

STILbench User Manual

STILbench
February 17, 2011

Contents

I	Virtual Tester User Manual	1
1	Introduction	1
1.1	Why should I use Virtual Test ?	1
1.2	Virtual Tester solution	1
2	Methodology	2
3	stilbench Setup File Options	3
3.1	stil_file	3
3.2	stil_pattern_exec	3
3.3	dut_name	3
3.4	precision	4
3.5	workdir	4
3.6	use_absolute_paths	4
3.7	fsdb_file_size	5
3.8	fsdb_file_count	5
4	Simulation results	6
5	Running the tool	7
5.1	Running STILbench	7
5.1.1	Usage	7
5.1.2	Setup file template	8
5.1.3	STILbench output files	9
6	Implementation	10
6.1	Linking to the device	10
6.2	The Results	11

Part I

Virtual Tester User Manual

1 Introduction

The Virtual Tester is part of of Test Insight closed loop design and test environment. The tool provides a link back from the production test program to the design environment. Using that link, provides the designer a tester like environment to verify the operation of the test program.

1.1 Why should I use Virtual Test ?

When the test engineer finally gets the silicon and starts debugging the test program, he needs to analyze failures. The test engineer in that stage, needs to understand the cause of failures which could be:

- A design issue - the test is initially created by the design team. The designer might have used methodologies that are incompatible with production test. For example, they might have used force to set a value to some internal DUT registers. Such operation is obviously not valid when running in production test environment. The designer, might not be aware of that.
- A conversion issue - the test engineer have converted the test from design event driven to tester which is cycle driven. The conversion process might have been faulty. For example, the conversion might have neglected some signal in the test vectors. In such case, the test will fail though the original test was accurate enough.
- A production issue - this is the real reason for testing - find faults that are mismatch between the design model of the DUT and the device as produced in silicon.

So, the test engineer needs to analyze where each fail on the tester belongs (design, conversion or production). Using virtual test, provides a tool to analyze and clear the first two fail reasons and do so *before silicon* and without using the tester. Emulating the tester environment in design simulation, reveals design and test compatibility issues as well as conversion issues. The test bench created by Virtual test, reflects the activity of the tester in design environment.

1.2 Virtual Tester solution

- Virtual Tester acts as tester thus reveals tester incompatibilities.
- Highlights bad conversion issues as test fails in simulation.
- Focus only on Model-Silicon mismatch.

2 Methodology

The Virtual Tester reads the following elements:

- Pins
- AC equations
- AC specs
- Test Vectors

and creates a Verilog language test bench emulating the tester operation. The Verilog test bench is linked in the design environment with the DUT model and simulated together. Input signals are driven as in the test vectors according to the tester timing. Output signals are compared during each compare window and the DUT data is compared to the test vectors expected data. A fail is when the DUT data does not match the vector expected data.

3 stilbench Setup File Options

The following sections describe all stilbench setup file options.

3.1 stil_file

Description

Selects STIL file to be translated into a Verilog testbench

Syntax

```
stil_file = file_name
```

Example

```
stil_file = 'simple.stil'
```

3.2 stil_pattern_exec

Description

Defines STIL PatternExec name.

First one is used if not specified.

Syntax

```
stil_pattern_exec = '<name>'
```

Example

```
stil_pattern_exec = 'Pattern1'
```

3.3 dut_name

Description

Name of the device model. (Optional)

Syntax

```
dut_name = '<name>'
```

Example

```
dut_name = 'Device'
```

3.4 *precision*

3.4 *precision*

Description

Simulation precision.

Syntax

```
precision = 'ns' | 'fs' | 'ps'
```

Example

```
precision = 'ps'
```

3.5 *workdir*

Description

VT output directory.

Syntax

```
workdir = "<directory_name>"
```

Example

```
workdir = "vt"
```

3.6 *use_absolute_paths*

Description

Use of absolute paths in VT files, true by default.

Syntax

```
use_absolute_paths = 0|1
```

Example

```
use_absolute_paths = 1
```

3.7 `fsdb_file_size`

3.7 `fsdb_file_size`

Description

Limit FSDB file size.

Unlimited by default.

Syntax

```
fsdb_file_size = <integer>
```

Example

```
fsdb_file_size = 12
```

3.8 `fsdb_file_count`

Description

Limit the number of FSDB files.

Syntax

```
fsdb_file_count = <number>
```

Example

```
fsdb_file_count = 3
```

4 Simulation results

The simulation results are produced in three forms:

1. A log file of the simulation, reporting execution of each test vector, fails and contentions. Contention is a situation that both the DUT and the tester model are driving the same signal. Log file example:

```
[ VTVECINF ][ vector information ][ pattern domain1_WR_RD_Mode0 ]
          [ vector #0 ][ wft T0 ][ time 0 ]
[ VTCMPVAL ][ compare failed ][ D0 in domain1_WR_RD_Mode0 ]
          [ vector #47 ][ wfc L ][ real z ][ exp 0 ][ time 19100000 ]
[ VTVECINF ][ vector information ][ pattern domain1_WR_RD_Mode0 ]
          [ vector #47 ][ wft T6 ][ time 18800000 ]
[ VTDRVCNT ][ contention between device and tester ][ signal D0 ]
          [ time 19040000 ]
```

2. A fail log reporting just failures.

```
[ VTCMPVAL ][ compare failed ][ D0 in domain1_WR_RD_Mode0 ]
          [ vector #51 ][ wfc H ][ real 0 ][ exp 1 ][ time 20532000 ]
```

3. A VCD trace with indication of all DUT signals, the timesets used, indication of the compare window and drive state of each signal, as seen on Figure 1.

Figure 1: Virtual Tester VCD Trace

5 Running the tool

The following two steps are required in order to get output simulation files:

1. Running ATEin - Receives test program files as input and outputs STIL files.
2. Running STILbench - Receives the STIL files resulting from the previous step and outputs VT.

In order to run the tool you need to :

1. Set up ATE2STIL environment:

```
setenv ATE2STILHOME /path/to/ate2stil/installation
source ATE2STILHOME /etc/ate2stilrc.csh
```

2. Run ATEin tool for the relevant source. See [ATEin user manual](#).
3. Run STILbench with the output STIL file from previous stage, as input to the STILbench .
4. Run simulation.

5.1 Running STILbench

5.1.1 Usage

Usage: `stilbench [-help] [-version] [-timestamp] [-licwait] [-setup setup_file.py] [-work_dir directory]`

Command	Description
-help	display this message
-version	show version number and exit
-timestamp	enable log timestamp
-licwait	wait for the license, if not available
-setup <str>	path to setup file The setup file is the main execution control it indicates the source platform, the target format and list of files for conversion
-work_dir <str>	working directory, default = '.'

5.1.2 Setup file template

Setup file template contains the following flags:

1. `stil_dir = "path"`
Optional flag, defines the STIL input files base directory. Paths specified in 'stil_files', if not absolute, will be relative to this directory.
2. `stil_files = [ 'path2', 'path2', 'pathN' ]`
Mandatory flag, defines the path (or list of paths) to the STIL input file(s). Define here the stil output files generated by ATEin tool in the previous stage.
3. `dut_name = "dut"`
Optional flag, defines the name of the device model. Default: `dut`.
4. `precision = "ps"`
Optional flag, defines simulation precision. Possible values are "ns", "ps" and "fs", default: `ps`.
5. `work_dir = "vt"`
Optional flag, defines the VT output directory, place to put the VT files. Default: `vt`.
6. `use_absolute_paths = 1`
Optional flag, enables use of absolute paths in VT files, true by default.

You can get a setup file template by using the following command:

```
ATE2STILHOME /bin/stilbench -template
```

You will get the following message :

The tool will generate template setup file in './vtgen_setup_template.py' and silently exit.

In order to create a setup file, you can use the setup file template and edit the relevant flags.

In order to run the STILbench tool, use the following command:

```
ATE2STILHOME /bin/stilbench -setup < Setup_Filename >
```

5.1.3 STILbench output files

The resulting from running STILbench are simulation files and Results per test. Creation of a verilog test-bench emulating the behavior of the tester considering timing pattern data. During the run time there is a creation of VCD trace of DUT signals and Tester activity. Test-bench is open for user customization. STILbench builds *vt* directory under project directory, name of directory defined in setup file(default = ./vt). The *vt* directory contains the following files:

File name	Description
<i>pattern_name</i> _pattern.bin	Binary pattern file, emulation of the tester pattern memory.
<i>pattern_name</i> _pattern_params.bin	Additional parameters to binary pattern file.
<i>pattern_name</i> .v	Test emulation module.
test_wrapper.v	Top level module, connecting the device top level to the tester module and defines the VCD signals.
<i>pattern_name</i> _timing.v	Tester timing file.
<i>pattern_name</i> _timing_control.v	Tester timing file.
board.hv	Board delay definitions, default:zero delay.
test.hv	Global definitions and timescale.
monitor.v	Defines control signals for each of the device signals.
tester.v	Tester engine main control.
vtrun1	Prototype for the Unix style shell script to compile and run the Verilog simulation, using explicit files.
vtrun2	Prototype for the Unix style shell script to compile and run the Verilog simulation, using verilog library option.

Under normal circumstances none of the above Verilog and binary files should be modified. The only exception is the top level (*test_wrapper.v*) file that defines tester to device connectivitya and delay definitions (*board.hv*) file. The *vtrun* script should be merged with a script running the normal device simulation.

6 Implementation

6.1 Linking to the device

The *vtrun* script should be merged with a script running the normal device simulation linking to the device.

Below is a *vtrun* file, after adding the link to the device model files marked with green:

```
test.hv
ATE2STILHOME /etc/verilog/vtw_stddef.hv
ATE2STILHOME /etc/verilog/vtw_wfgdef.hv
ATE2STILHOME /etc/verilog/clockgen.v
ATE2STILHOME /etc/verilog/vtw_logger.v
ATE2STILHOME /etc/verilog/vtw_wfg.v
board.hv
domain1.WR_RD_Mode0_timing.v
domain1.WR_RD_Mode0_test.v
domain1.WR_RD_Mode0_timing_control.v
test_J750_ate2stil_setup.v
tester.v
monitor.v
../Device_files/CNTREG.v
../Device_files/DOWNCNTR.v
../Device_files/I8253.v
../Device_files/MODEREG.v
../Device_files/OUTCTRL.v
../Device_files/OUTLATCH.v
../Device_files/READ.v
```

After editing the *vtrun* you need to:

1. Compile the DUT model with the Test Bench
2. Run the simulation

6.2 The Results

After running simulation, the *vt* results directory will contain the following files:

File name	Description
vtw.fail	Log of failures for the test. Empty if the test passes.
vtw.log	Simulation execution log file.
vtw.warn	Log of warnings.
<i>pattern.name.vcd</i>	A <i>VCD</i> formatted trace of the test, showing all signals of the device and indicators of failures, strobe windows and direction. Some additional useful information is present, including timeset, vector number.